

Image Captioning mit Flickr8k

Zielsetzung

Ziel dieser Mini-Challenge ist die Implementierung, das Training und die Evaluation verschiedener Deep-Learning-Architekturen zur automatischen Generierung von Bildbeschreibungen (Image Captioning) auf dem Flickr8k-Datensatz. Im Fokus steht insbesondere die Untersuchung des Einflusses von Attention-Mechanismen auf die Qualität und Interpretierbarkeit der generierten Captions.

Es werden **zwei zentrale Modellvarianten** verglichen, ergänzt durch eine explorative Erweiterung:

Modell	Ansatz	Referenz
CNN-LSTM Baseline	Globaler Bildvektor → LSTM-Decoder	Vinyals et al., 2015
Attention-Modell	Räumliche Feature-Map + Soft-Attention	Xu et al., 2015
Transformer-Decoder	Cross-Attention auf Bildpatches (explorativ)	–

Alle Modelle werden mit **Python**, **PyTorch** und **PyTorch Lightning** umgesetzt.

1 Datensatz und Vorbereitung

1.1 Flickr8k-Datensatz

Eigenschaft	Details
Umfang	8 000 Bilder aus verschiedenen Alltagsszenen
Annotationen	5 manuelle Bildbeschreibungen pro Bild
Format	Bilder: JPG Captions: Textdateien
Training	ca. 6 000 Bilder
Validierung	ca. 1 000 Bilder
Test	ca. 1 000 Bilder

1.2 Vorverarbeitung

Bildverarbeitung

- Skalierung auf feste Grösse (224×224 px)
- Normalisierung der Pixelwerte
- **Datenaugmentierung (obligatorisch)**: HorizontalFlip, Farbvariationen, Random Crop u.a. – besonders wichtig wegen des kleinen Datensatzes
- Visueller Encoder: vortrainiertes CNN (voraussichtlich ResNet-50)

- Da Augmentierung zur Trainingszeit angewendet wird, entfällt das vorab Abspeichern der Bildfeatures – das CNN wird bei jedem Schritt auf dem augmentierten Bild ausgeführt

Merkmalsdarstellung

- **Baseline:** globaler Bildfeature-Vektor
- **Attention- und Transformer-Variante:** räumliche Feature-Map der letzten CNN-Schicht (z.B. $14 \times 14 \times D$) – räumliche Informationen bleiben erhalten

Textverarbeitung

- Tokenisierung und Umwandlung in Kleinbuchstaben
- Bereinigung von Sonderzeichen
- Ergänzung mit <start>- und <end>-Tokens
- Kürzere Sequenzen werden mit <pad>-Tokens auf die maximale Sequenzlänge im Batch aufgefüllt
- Seltene Wörter → <unk>-Token

Feedback Dozent: Datenaugmentierung ist obligatorisch (kleines Datenset). Als Konsequenz werden Bildfeatures nicht mehr vorab gespeichert, sondern bei jedem Trainingsschritt aus dem augmentierten Bild berechnet. Zusätzlich wurde <pad> als Special Token ergänzt.

Datenstruktur: Für jedes Bild entstehen mehrere Trainingsbeispiele der Form:

(Bildfeatures, Eingabesequenz → Zielsequenz) | Datenverwaltung via PyTorch-Lightning-DataModule

2 Baseline-Modell: CNN–LSTM (Show and Tell)

2.1 Architektur

Komponente	Beschreibung
Encoder	Vortrainiertes CNN → globaler Feature-Vektor → lineare Projektion → LSTM-Initialisierung
Decoder	LSTM generiert Caption Wort für Wort (Eingabe: Wortembedding + Hidden-State)
Ausgabe	Softmax-Wahrscheinlichkeitsverteilung über das Vokabular
Teacher Forcing	Ground-Truth-Tokens als Eingabe für den nächsten Zeitschritt
Loss	Cross-Entropy-Loss (negative Log-Likelihood) über alle Tokens

2.2 Training & Evaluation

Hyperparameter	Evaluation	Inference
Optimierer: Adam Lernrate: ca. $1e-4$ Batch-Grösse: ca. 64 Epochen: 20–30	Primär: CLIPScore Klassisch: BLEU-1 bis BLEU-4 CLIPScore bewertet semantische Übereinstimmung direkter als n-Gram-Metriken.	Greedy Decoding Beam Search (Beam Size 3–5)

3 Attention-Modell: Show, Attend and Tell

3.1 Architektur

Komponente	Beschreibung
------------	--------------

Encoder	Räumliche CNN-Feature-Map (statt globalem Vektor) – jedes Patch repräsentiert einen lokalen Bildbereich
Soft-Attention	Kontextvektor = gewichtete Summe der Bildpatches; Gewichte bestimmen Fokusbereich pro Zeitschritt
Decoder	LSTM erhält: Wortembedding + Kontextvektor + Hidden-State; Ausgabe via Softmax
Implementierung	Ausschliesslich deterministische Soft-Attention

3.2 Training & Evaluation

Training identisch zur Baseline (Teacher Forcing, Cross-Entropy-Loss). Zusätzliche Auswertungsschritte:

- Visualisierung von Attention-Heatmaps
- Qualitative Caption-Analyse
- Untersuchung: semantisch bessere Captions? Fokus auf sinnvolle Bildbereiche?

4 Explorative Erweiterung: Transformer-Decoder

Motivation: Explorative Untersuchung, ob ein kompakter Transformer-Decoder ohne LSTM auf dem kleinen Flickr8k-Datensatz sinnvoll trainiert werden kann. Dient als moderne Erweiterung, nicht als primärer Vergleich.

4.1 Architektur

Komponente	Beschreibung
Encoder	Räumliche CNN-Features + zusätzliche Positionskodierungen
Transformer-Blöcke	Masked Self-Attention Cross-Attention auf Bildpatches Feed-Forward-Netze
Eingaben	Wortembeddings + Positionskodierungen; Cross-Attention auf Bildfeatures
Generierung	Autoregressive Caption-Generierung

4.2 Training & Evaluation

Geplante Modellgrösse (kompakt):

Decoder-Layer	Attention Heads	Embedding-Dim
2–4	4–8	reduziert

Evaluation: CLIPScore, BLEU-4, qualitative Analyse + Visualisierung von Cross-Attention-Maps.

Forschungsfrage: Kann ein rein attentionbasierter Decoder auf einem kleinen Datensatz ohne vortrainiertes Sprachmodell sinnvoll lernen?

5 Implementierungsplan mit PyTorch Lightning

Modul	Verantwortlichkeiten
DataModule	Laden von Bildern und Captions Tokenisierung Batch-Erstellung Train/Val/Test-Splits CNN-Feature-Laden
LightningModule Baseline	CNN-Encoder + LSTM-Decoder Loss BLEU- und CLIPScore-Berechnung Train/Val/Test-Steps

LightningModule Attention	Erweiterung der Baseline um Attention-Mechanismus Kontextvektorberechnung Speicherung der Attention-Gewichte
LightningModule Transformer	Transformer-Decoder Masked Self-Attention Cross-Attention Positionskodierungen Cross-Attention-Visualisierung
Training (gemeinsam)	PyTorch Lightning Trainer Early Stopping LR-Scheduler GPU-/MPS-Beschleunigung Optional: W&B Logging

5.6 Präsentation und Resultate

Die Resultate werden in einem gut dokumentierten **Jupyter-Notebook** präsentiert:

Quantitativ	Qualitativ	Visualisierungen
CLIPScore BLEU-1 bis BLEU-4 Modellvergleich	Caption-Vergleiche Greedy vs Beam Search Fehleranalyse	Attention-Heatmaps Cross-Attention-Maps Feature-Visualisierung

Fokus der Auswertung: Einfluss von Attention | Interpretierbarkeit der Modelle | Unterschiede zwischen rekurrenten und rein attentionbasierten Decodern